

LINUX CLI

MASTERY

POSTING EDITION / COMPLETE / PRACTICAL / INTERVIEW-ORIENTED

SOURCE	Yeshwanth-Gujja/Linux_CLI_Mastery repository
COVERAGE	17 core learning notes / cheatsheet / examples / troubleshooting / interview preparation
SCOPE	Linux command line, shell, Bash, administration, networking, storage, automation
DESIGN	Clickable navigation / internal bookmarks / URL hyperlinks / readable code blocks

CORE LEARNING MODEL

problem → command → arguments/options → input → output/status → composition → verification

This edition is designed as a practical Linux CLI book: build the mental model, operate safely, compose commands, administer systems, automate with Bash, troubleshoot methodically, and prepare for interviews.

POSTING NOTE

This PDF is a consolidated presentation of the supplied Linux_CLI_Mastery repository. The repository structure and terminology are retained rather than replaced with unrelated material.

Contents

The numbered notes are presented in the repository learning order.

Linux CLI — Foundations

Linux CLI — Filesystem and Paths

Linux CLI — Files and Directories

Linux CLI — Search and Text Processing

Linux CLI — Streams, Pipes and Redirection

Linux CLI — Shell Environment and Command Control

Linux CLI — Permissions, Ownership and Security

Linux CLI — Users, Groups and Identity

Linux CLI — Processes, Jobs and Signals

Linux CLI — System Information and Storage

Linux CLI — Networking and Remote Access

Linux CLI — Packages, Services, Scheduling and Archives

Linux CLI — Editors and Shell Configuration

Linux CLI — Bash Scripting

Linux CLI — Advanced CLI Tools and Reference

Linux CLI — Troubleshooting, Interview and Learning

Linux CLI Mastery — Deep Dives and Gaps Filled

Appendices

Linux CLI Cheatsheet

Practice Examples



Learning Map

Linux CLI Mastery — Learning Map

Start Here

This repository is a practical Linux Command-Line Interface reference built from the supplied Linux CLI notes and expanded with additional topics that are commonly required for real development, DevOps, administration, troubleshooting, and interviews.

Recommended order

- 1 [Foundations](01-foundations.md)
- 2 [Filesystem and Paths](02-filesystem-and-paths.md)
- 3 [Files and Directories](03-files-and-directories.md)
- 4 [Search and Text Processing](04-search-and-text-processing.md)
- 5 [Streams, Pipes and Redirection](05-streams-pipes-and-redirection.md)
- 6 [Shell Environment and Command Control](06-shell-environment-and-command-control.md)
- 7 [Permissions, Ownership and Security](07-permissions-ownership-and-security.md)
- 8 [Users, Groups and Identity](08-users-groups-and-identity.md)
- 9 [Processes, Jobs and Signals](09-processes-jobs-and-signals.md)
- 10 [System Information and Storage](10-system-information-and-storage.md)
- 11 [Networking and Remote Access](11-networking-and-remote-access.md)
- 12 [Packages, Services, Scheduling and Archives](12-packages-services-scheduling-and-archives.md)
- 13 [Editors and Shell Configuration](13-editors-and-shell-configuration.md)
- 14 [Bash Scripting](14-bash-scripting.md)
- 15 [Advanced CLI Tools and Reference](15-advanced-cli-tools-and-reference.md)
- 16 [Troubleshooting, Interview and Learning](16-troubleshooting-interview-and-learning.md)
- 17 [Deep Dives](17-deep-dives.md)

Skill progression

Level 1 — Operate

`pwd` → `ls` → `cd` → `mkdir` → `touch` → `cp` → `mv` → `rm` → `cat` → `less`

Level 2 — Investigate

`find` → `grep` → `file` → `stat` → `head` → `tail` → `sort` → `uniq` → `wc`

Level 3 — Compose

Pipes → redirection → command substitution → quoting → exit status → `xargs` → `sed` → `awk`

Level 4 — Administer

Permissions → ownership → users/groups → processes → signals → services → logs → packages → storage

Level 5 — Operate remotely

Networking → DNS → sockets → SSH → SCP → rsync → HTTP/API inspection



Level 6 — Automate

Bash variables → parameters → conditions → loops → functions → safe scripting → scheduling

Level 7 — Troubleshoot

Identify → inspect → isolate → verify → read logs → test dependencies → fix → re-test → document

How to study each topic

For every command, learn four things:

- 1 What problem does it solve?
- 2 What input does it consume?
- 3 What output/status does it produce?
- 4 How can it be safely combined with other commands?

Do not optimize for memorizing hundreds of flags. Build the mental model first, then use `man`, `--help`, and examples as reference tools.

YESHWANTH-GUJJA

Linux CLI — Foundations

Note — This section is part of Linux CLI Mastery. Source material from the supplied Linux CLI notes is preserved and organized by topic.

1. Linux CLI Fundamentals

1.1 What is Linux?

Linux is an open-source Unix-like operating system kernel.

A complete Linux operating system is typically built from:

```
Linux Kernel
+
System Libraries
+
System Utilities
+
Shell
+
Applications
```

Examples of Linux distributions:

```
Ubuntu
Debian
Fedora
Arch Linux
Linux Mint
RHEL
Rocky Linux
AlmaLinux
openSUSE
```

2. What is CLI?

CLI stands for Command-Line Interface.

It allows users to interact with an operating system by entering commands into a terminal.

Example:

```
ls
```

Instead of clicking a file manager, you can use:

```
cd Documents
ls
```

3. Terminal vs Shell

These are not the same thing.

Terminal



A terminal is an interface through which you interact with a shell.

Examples:

```
GNOME Terminal  
Konsole  
Windows Terminal  
Alacritty
```

Shell

A shell is a program that interprets commands and executes them.

Common shells:

```
bash  
zsh  
fish  
sh  
ksh
```

Most Linux beginners encounter Bash first.

4. Bash

Bash stands for:

```
Bourne Again SHell
```

It is one of the most widely used Unix/Linux shells.

Check your current shell:

```
echo "$SHELL"
```

Check the current shell process:

```
ps -p $$ -o comm=
```

5. Command Prompt

A typical prompt might look like:

```
yeshwanth@ubuntu:~$
```

Breakdown:

```
yeshwanth → username  
@         → separator  
ubuntu    → hostname  
:         → separator  
~         → current directory  
$         → normal user prompt
```

A root prompt commonly ends with:

```
#
```

Example:

```
root@ubuntu:/#
```

Do not assume `$` or `#` is part of the command.

If the prompt shows:

```
$ ls
```

you type:

```
ls
```

6. Root User

The root user is the superuser.

Root has extensive privileges over the system.

Example:

```
whoami
```

Normal user:

```
yeshwanth
```

Root:

```
root
```

Avoid using root unnecessarily.

7. sudo

`sudo` allows an authorized user to execute a command with elevated privileges.

Example:

```
sudo apt update
```

Another example:

```
sudo systemctl restart nginx
```

Check whether your account can use sudo:

```
sudo -l
```

8. Basic Command Structure

A command commonly follows:

```
command [options] [arguments]
```

Example:

```
ls -lah /home
```

Breakdown:

```
ls      → command  
-lah    → options  
/home   → argument
```

9. Options

Options modify command behavior.

Example:

```
ls
```

versus:

```
ls -l
```

Long-form options often use:

```
ls --all
```

Short options commonly use:

```
-a
```

Multiple short options can often be combined:

```
ls -lah
```

10. Arguments

Arguments tell the command what to operate on.

Example:

```
cat notes.txt
```

Here:

```
cat      → command  
notes.txt → argument
```

Another example:

```
rm file.txt
```



11. Command History

View previous commands:

```
history
```

Run the previous command:

```
!!
```

Run command number 25:

```
!25
```

Run the most recent command beginning with **git**:

```
!git
```

12. Command History Navigation

Press:

```
↑
```

to move backward through previous commands.

Press:

```
↓
```

to move forward.

Useful shortcuts:

```
Ctrl + P → previous command  
Ctrl + N → next command
```

13. Clearing the Terminal

```
clear
```

Keyboard shortcut:

```
Ctrl + L
```

14. Exiting the Shell

```
exit
```

or:

```
Ctrl + D
```

15. Getting Help

Linux provides extensive command documentation.

man

```
man ls
```

Displays the manual page for `ls`.

Example:

```
man chmod
```

16. --help

Many commands support:

```
ls --help
```

This provides a shorter usage reference.

17. info

Some GNU utilities provide detailed documentation through:

```
info coreutils
```

18. whatis

Provides a short description.

```
whatis ls
```

19. apropos

Searches manual-page descriptions.

```
apropos password
```

Useful when you know what you want to accomplish but don't know the command.

20. Finding Command Location

which

```
which python
```

Shows the executable found through the current **PATH**.

type

```
type ls
```

This is often more informative.

Example:

```
type cd
```

may reveal that **cd** is a shell builtin.

21. whereis

```
whereis python
```

Searches for binary, source, and manual locations associated with a command.

Linux CLI — Filesystem and Paths

Note — This section is part of Linux CLI Mastery. Source material from the supplied Linux CLI notes is preserved and organized by topic.

22. Linux Filesystem

Linux uses a hierarchical filesystem.

The root of the filesystem is:

```
/
```

Important directories:

```
/
├── bin
├── boot
├── dev
├── etc
├── home
├── lib
├── media
├── mnt
├── opt
├── proc
├── root
├── run
├── sbin
├── srv
├── sys
├── tmp
├── usr
└── var
```

23. Root Directory /

```
cd /
```

/ is the filesystem root.

Do not confuse:

```
/
```

with:

```
/root
```

/root is the root user's home directory.

24. /home

Contains normal users' home directories.

Example:



```
/home/yeshwanth  
/home/alice  
/home/bob
```

25. /root

Home directory of the root user.

```
/root
```

It is different from:

```
/home/root
```

26. /etc

Contains system configuration files.

Examples:

```
/etc/hosts  
/etc/passwd  
/etc/group  
/etc/ssh/  
/etc/systemd/
```

27. /var

Contains variable data.

Examples:

```
/var/log  
/var/cache  
/var/lib  
/var/tmp
```

Logs are commonly stored under:

```
/var/log
```

28. /tmp

Temporary files.

```
/tmp
```

Do not assume files in `/tmp` are permanent.

29. /usr

Contains many user-space programs, libraries, and shared resources.

Common directories:

```
/usr/bin  
/usr/sbin  
/usr/lib  
/usr/share
```

30. /bin

Traditionally contains essential user commands.

On many modern distributions, `/bin` may be a symbolic link to:

```
/usr/bin
```

31. /sbin

Traditionally contains system administration commands.

Modern distributions may merge this into `/usr/bin` through symlinks.

32. /dev

Contains device files.

Examples:

```
/dev/null  
/dev/zero  
/dev/random  
/dev/sda
```

33. /proc

A virtual filesystem exposing process and kernel information.

Example:

```
ls /proc
```

Current process information:

```
ls /proc/$$
```

34. /sys

Provides information and interfaces related to devices, drivers, and the kernel.

35. /boot

Contains files required for booting the system.

Examples may include:

```
vmlinuz  
initramfs  
grub/
```

36. /media

Common mount location for removable media.

37. /mnt

Common temporary mount location.

38. /opt

Often used for optional/additional software packages.

39. /srv

Data served by system services.

40. /run

Runtime data created since boot.

41. Current Working Directory

Use:

```
pwd
```

`pwd` means:

```
Print Working Directory
```

Example:

```
/home/yeshwanth/projects
```

42. `cd`

Change directory.

```
cd /home
```

43. Home Directory

```
cd ~
```

or simply:

```
cd
```

`~` represents the current user's home directory.

44. Parent Directory

```
cd ..
```

`..` means the parent directory.

45. Current Directory

```
.
```

Example:

```
./script.sh
```

means execute `script.sh` from the current directory.

46. Root Path vs Relative Path

Absolute:

```
cd /home/yeshwanth/projects
```

Relative:

```
cd projects
```

An absolute path begins at `/`.

A relative path is interpreted from the current working directory.

YESHWANTH-GUJJA

Linux CLI — Files and Directories

Note — This section is part of Linux CLI Mastery. Source material from the supplied Linux CLI notes is preserved and organized by topic.

47. `ls`

Lists directory contents.

```
ls
```

48. `ls -l`

Long listing:

```
ls -l
```

Example:

```
-rw-r--r-- 1 yesh yesh 1200 Sep 30 10:00 notes.txt
```

49. `ls -a`

Shows hidden files.

```
ls -a
```

Hidden filenames normally begin with:

```
.
```

Examples:

```
.bashrc  
.gitconfig  
.ssh
```

50. `ls -h`

Human-readable sizes:

```
ls -lh
```

51. Useful `ls` Combinations

```
ls -lah
```

Means:

```
-l → long format  
-a → hidden files  
-h → human-readable sizes
```

52. Creating Directories

```
mkdir projects
```

53. Nested Directories

```
mkdir -p projects/linux/notes
```

-p creates missing parent directories.

54. Creating Files

Common approaches include:

```
touch notes.txt
```

touch creates an empty file if it does not exist.

It can also update timestamps on an existing file.

55. **cat**

Display file contents:

```
cat notes.txt
```

56. **less**

View large files page by page:

```
less notes.txt
```

Useful controls:

```
Space → next page  
b      → previous page  
/word  → search  
n      → next match  
q      → quit
```

57. more

Another pager:

```
more notes.txt
```

`less` is generally more capable.

58. head

Display the beginning of a file.

```
head notes.txt
```

Default behavior commonly shows the first 10 lines.

Specify lines:

```
head -n 20 notes.txt
```

59. tail

Display the end of a file.

```
tail notes.txt
```

Specify lines:

```
tail -n 20 notes.txt
```

60. tail -f

Follow a growing file:

```
tail -f /var/log/app.log
```

Commonly used for monitoring logs.

Exit:

```
Ctrl + C
```

61. Copying Files

```
cp source.txt destination.txt
```

62. Copying Directories

Use recursive copying:

```
cp -r source_dir destination_dir
```

63. Moving Files

```
mv old.txt new.txt
```

Can rename a file.

64. Moving a File to Another Directory

```
mv file.txt documents/
```

65. Renaming a Directory

```
mv old_directory new_directory
```

66. Removing Files

```
rm file.txt
```

Be careful: **rm** normally does not provide a recycle-bin mechanism.

67. Removing Directories

Empty directory:

```
rmdir emptydir
```

Recursive removal:

```
rm -r directory
```

Force:

```
rm -rf directory
```

rm -rf is extremely powerful and dangerous.

Always verify the path before executing it.

68. Wildcards

*

Matches many characters.

```
ls *.txt
```

Matches:

```
a.txt
notes.txt
hello.txt
```

?

Matches one character.

```
ls ?.txt
```

Matches:

```
a.txt
b.txt
```

but not:

```
notes.txt
```

69. Character Classes

```
ls [abc].txt
```

Matches:

```
a.txt
b.txt
c.txt
```

Range:

```
ls [a-z].txt
```

70. Hidden Files

A filename beginning with `.` is conventionally hidden.

Examples:

```
.bashrc
.profile
.git
```

Show them:

```
ls -la
```

71. File Types

Linux uses a unified filesystem model.

Common file types:

```
- regular file
d directory
l symbolic link
c character device
b block device
p named pipe
s socket
```

Check with:

```
ls -l
```

72. file

Determine what kind of file something is:

```
file notes.txt
```

Example:

```
notes.txt: ASCII text
```

73. File Extensions

Linux does not require extensions to identify file types.

For example:

```
script
script.sh
```

can both be executable files.

The extension is primarily a naming convention.

74. Symbolic Links

Create a symbolic link:

```
ln -s target linkname
```

Example:



```
ln -s /var/log/app.log app.log
```

A symbolic link stores a path/reference to another file.

75. Hard Links

Create:

```
ln original.txt hardlink.txt
```

A hard link refers to the same underlying inode/data as the original file.

Important distinction:

```
Symbolic link → points to a path
Hard link      → points to the same inode
```

YESHWANTH-GUJJA

Linux CLI — Search and Text Processing

Note — This section is part of Linux CLI Mastery. Source material from the supplied Linux CLI notes is preserved and organized by topic.

76. Inodes

An inode stores filesystem metadata about a file, such as:

- File type
- Permissions
- Owner
- Group
- Size
- Timestamps
- Link count
- Pointers/references to file data

Filename information is associated with directory entries rather than being the inode itself.

77. stat

Display detailed file metadata:

```
stat notes.txt
```

78. File Timestamps

Important timestamps include:

```
atime → access time  
mtime → modification time  
ctime → metadata/status change time
```

On Linux filesystems, **ctime** is not creation time.

Some filesystems also expose birth time when supported.

79. touch

Create/update timestamps:

```
touch file.txt
```

Set a specific modification/access time:

```
touch -t 202609301200 file.txt
```



80. Searching Files with `find`

Basic:

```
find . -name "notes.txt"
```

Search recursively from the current directory.

81. Find by Extension

```
find . -name "*.txt"
```

82. Find by Type

Files:

```
find . -type f
```

Directories:

```
find . -type d
```

Symbolic links:

```
find . -type l
```

83. Find by Size

```
find . -size +100M
```

Find files larger than approximately 100 MB.

84. Find by Modification Time

```
find . -mtime -1
```

Files modified within the last day.

85. Find and Execute

```
find . -name "*.log" -exec wc -l {} \;
```

`{}` is replaced by each matching path.

86. locate

```
locate notes.txt
```

Uses a prebuilt database.

It is usually faster than `find`, but the database may not contain the newest files.

87. grep

Search text inside files.

```
grep "error" log.txt
```

88. Case-Insensitive Search

```
grep -i "error" log.txt
```

89. Recursive Search

```
grep -r "TODO" .
```

Searches recursively.

90. Show Line Numbers

```
grep -n "error" log.txt
```

91. Invert Match

```
grep -v "debug" log.txt
```

Shows lines that do not match.

92. Count Matches

```
grep -c "error" log.txt
```

93. Extended Regular Expressions



```
grep -E "error|warning" log.txt
```

Searches for either **error** or **warning**.

94. **grep** with Pipes

```
ps aux | grep nginx
```

A common CLI pattern.

95. **wc**

Word/line/byte counting.

```
wc file.txt
```

Options:

```
wc -l file.txt  
wc -w file.txt  
wc -c file.txt
```

Meaning:

```
-l → lines  
-w → words  
-c → bytes
```

96. **sort**

Sort lines:

```
sort names.txt
```

Reverse:

```
sort -r names.txt
```

Numeric:

```
sort -n numbers.txt
```

97. **uniq**

Remove adjacent duplicate lines:

```
uniq names.txt
```

Usually combine with **sort**:



```
sort names.txt | uniq
```

Count duplicates:

```
sort names.txt | uniq -c
```

98. cut

Extract fields or characters.

Example:

```
cut -d: -f1 /etc/passwd
```

Meaning:

```
-d: → delimiter is :  
-f1 → select field 1
```

99. tr

Translate or delete characters.

Convert lowercase to uppercase:

```
echo "hello" | tr 'a-z' 'A-Z'
```

100. tee

Reads standard input and writes it both to standard output and a file.

```
echo "Hello" | tee output.txt
```

Append:

```
echo "World" | tee -a output.txt
```

Linux CLI — Streams, Pipes and Redirection

Note — This section is part of Linux CLI Mastery. Source material from the supplied Linux CLI notes is preserved and organized by topic.

101. Standard Streams

Every normal process has three standard streams:

```
0 → stdin  
1 → stdout  
2 → stderr
```

102. Standard Input

```
stdin
```

Usually comes from the keyboard.

103. Standard Output

```
stdout
```

Normally appears in the terminal.

104. Standard Error

```
stderr
```

Used for error/diagnostic output.

105. Output Redirection

Write output to a file:

```
ls > files.txt
```

> overwrites the file.

106. Append Output

```
ls >> files.txt
```

>> appends instead of replacing existing content.



107. Redirect Standard Error

```
command 2> errors.txt
```

108. Redirect Both Output and Error

Modern Bash:

```
command > output.txt 2>&1
```

Or:

```
command &> output.txt
```

The latter is Bash-specific.

109. Discard Output

```
command > /dev/null
```

Discard errors:

```
command 2> /dev/null
```

Discard both:

```
command > /dev/null 2>&1
```

110. `/dev/null`

`/dev/null` is a special device that discards data written to it.

Think:

```
/dev/null = data sink
```

111. `/dev/zero`

Produces a stream of zero bytes.

```
head -c 10 /dev/zero
```

112. `/dev/random` and `/dev/urandom`



Provide kernel-generated random data.

Modern Linux applications commonly use:

```
/dev/urandom
```

for non-blocking random data.

113. Pipes

A pipe sends one command's standard output to another command's standard input.

Syntax:

```
command1 | command2
```

Example:

```
ls | grep ".txt"
```

114. Multiple Pipes

```
cat access.log | grep "404" | sort | uniq -c
```

Concept:

```
cat
↓
grep
↓
sort
↓
uniq
```

115. Command Substitution

Execute a command and use its output:

```
echo "Today is $(date)"
```

Another form:

```
echo "Today is `date`"
```

The `$( )` form is preferred because it nests cleanly.

Linux CLI — Shell Environment and Command Control

Note — This section is part of Linux CLI Mastery. Source material from the supplied Linux CLI notes is preserved and organized by topic.

116. Variables

Create a shell variable:

```
name="Yeshwanth"
```

Important:

```
name="Yeshwanth"
```

Correct.

```
name = "Yeshwanth"
```

Incorrect in shell syntax because spaces change the command structure.

117. Accessing Variables

```
echo "$name"
```

118. Environment Variables

Environment variables are exported to child processes.

```
export NAME="Yeshwanth"
```

Check:

```
echo "$NAME"
```

119. env

Display environment variables:

```
env
```

120. printenv



```
printenv
```

Specific variable:

```
printenv PATH
```

121. PATH

PATH tells the shell where to search for executable commands.

```
echo "$PATH"
```

Example:

```
/usr/local/bin:/usr/bin:/bin
```

Directories are separated by:

```
:
```

122. Why ./command?

Suppose:

```
script.sh
```

is in the current directory.

Running:

```
script.sh
```

may fail because `.` may not be in **PATH**.

Use:

```
./script.sh
```

This explicitly refers to the current directory.

123. Quoting

Double quotes

```
echo "Hello $name"
```

Variables are expanded.

Single quotes



```
echo 'Hello $name'
```

The `$name` is treated literally.

124. Escaping

Use `\` to escape special interpretation.

Example:

```
echo "Price: \$100"
```

Output:

```
Price: $100
```

125. Command Chaining

Run commands sequentially:

```
command1 ; command2
```

The second command runs regardless of whether the first succeeds.

126. `&&`

```
command1 && command2
```

The second command runs only if the first succeeds.

127. `||`

```
command1 || command2
```

The second command runs if the first fails.

128. Exit Status

Linux commands return an exit status.

Convention:

```
0 → success  
non-zero → failure/error
```

Check the previous command:

```
echo $?
```

Example:

```
true  
echo $?
```

returns:

```
0
```

129. true and false

```
true
```

returns success.

```
false
```

returns failure.

These are useful in shell scripting and testing.

Linux CLI — Permissions, Ownership and Security

Note — This section is part of Linux CLI Mastery. Source material from the supplied Linux CLI notes is preserved and organized by topic.

130. Permissions

Linux permissions control access to files and directories.

Three basic permission types:

```
r → read
w → write
x → execute
```

Three basic ownership categories:

```
u → user/owner
g → group
o → others
```

131. Reading `ls -l`

Example:

```
-rwxr-xr-- 1 yesh developers 1200 Sep 30 10:00 script.sh
```

Breakdown:

```
-          → regular file
rwx       → owner permissions
r-x       → group permissions
r--       → others permissions
yesh      → owner
developers → group
1200      → size
```

132. Directory Permissions

For directories:

```
r → list directory entries
w → create/delete/rename entries
x → access/traverse the directory
```

This is an important distinction.

A directory's `x` permission is not "execute the directory."

It means traversal/search access.

133. `chmod`

Change permissions.

Symbolic:

```
chmod u+x script.sh
```

Adds execute permission for the owner.

134. Remove Permission

```
chmod u-x script.sh
```

135. Group Permission

```
chmod g+w file.txt
```

136. Others Permission

```
chmod o-r file.txt
```

137. Numeric Permissions

Permissions have numeric values:

```
r = 4  
w = 2  
x = 1
```

Therefore:

```
rxw = 7  
rw- = 6  
r-x = 5  
r-- = 4  
-wx = 3  
-w- = 2  
--x = 1  
--- = 0
```

138. chmod 755

```
chmod 755 script.sh
```

Means:

```
Owner  → rwx → 7  
Group  → r-x → 5  
Others → r-x → 5
```

139. chmod 644

```
chmod 644 file.txt
```

Means:

```
Owner  → rw-  
Group  → r--  
Others → r--
```

140. Recursive Permissions

```
chmod -R 755 directory/
```

Be careful with recursive permission changes.

Blindly applying **755** to an entire project can make files executable unnecessarily.

141. chown

Change ownership.

```
sudo chown user file.txt
```

Change owner and group:

```
sudo chown user:group file.txt
```

142. chgrp

Change group ownership:

```
sudo chgrp developers file.txt
```

143. Recursive Ownership

```
sudo chown -R user:group directory/
```

Again, verify the path before applying recursively.

144. Default Permissions and umask

`umask` controls which permission bits are removed when new files/directories are created.

Check:

```
umask
```

Example:

```
0022
```

The exact resulting permissions depend on the program's creation mode and the `umask`.

145. Special Permissions

Linux also has:

```
setuid
setgid
sticky bit
```

146. Setuid

Setuid on an executable causes the program to execute with the effective user ID of the file owner.

Numeric value:

```
4000
```

Example representation:

```
-rwsr-xr-x
```

This is security-sensitive.

147. Setgid

On executables, `setgid` affects effective group privileges.

On directories, `setgid` causes newly created files/subdirectories to inherit the directory's group in common Unix/Linux semantics.

Numeric value:

```
2000
```

148. Sticky Bit

Commonly used on shared directories such as:

```
/tmp
```

It prevents users from deleting/renaming files they do not own in a writable shared directory, subject to directory ownership and privilege rules.

Numeric value:

```
1000
```

149. umask, chmod, chown

Remember:

```
umask → default permission restriction  
chmod → permission bits  
chown → owner/group ownership
```

YESHWANTH-GUJJA

Linux CLI — Users, Groups and Identity

Note — This section is part of Linux CLI Mastery. Source material from the supplied Linux CLI notes is preserved and organized by topic.

150. Users

Show current user:

```
whoami
```

Show user identity and groups:

```
id
```

151. who

Show logged-in users:

```
who
```

152. w

Shows logged-in users and activity:

```
w
```

153. users

Simple list of logged-in usernames:

```
users
```

154. /etc/passwd

Contains user account information.

View:

```
cat /etc/passwd
```

A typical entry resembles:

```
username:x:1000:1000:User Name:/home/username:/bin/bash
```

Important fields:



```
username  
password placeholder  
UID  
GID  
GECOS/comment  
home directory  
login shell
```

Modern Linux systems do not normally store actual password hashes in `/etc/passwd`.

155. `/etc/shadow`

Contains password hashes and password-aging information.

```
sudo cat /etc/shadow
```

Access is restricted because it contains sensitive authentication information.

156. `/etc/group`

Contains group definitions.

```
cat /etc/group
```

157. User ID

Check:

```
id username
```

UID identifies the user.

158. Group ID

GID identifies the primary group.

159. Creating Users

On many Linux distributions:

```
sudo useradd username
```

A more interactive/high-level command may be:

```
sudo adduser username
```

The exact behavior differs by distribution.

160. Setting Password

```
sudo passwd username
```

161. Deleting Users

```
sudo userdel username
```

Remove the home directory too:

```
sudo userdel -r username
```

Use carefully.

162. Creating Groups

```
sudo groupadd developers
```

163. Adding User to Group

On many Linux systems:

```
sudo usermod -aG developers username
```

Important:

```
-a → append  
-G → supplementary groups
```

Without **-a**, you may replace the user's supplementary group list.

164. Viewing Groups

```
groups
```

or:

```
id
```

165. Switching User

```
su - username
```

The **-** requests a login shell environment.

166. **sudo** vs **su**

```
sudo command
```

Runs a command with elevated privileges according to sudo policy.

```
su - username
```

Switches to another user's account.

YESHWANTH-GUJJA

Linux CLI — Processes, Jobs and Signals

Note — This section is part of Linux CLI Mastery. Source material from the supplied Linux CLI notes is preserved and organized by topic.

167. Processes

A process is a running instance of a program.

Every process has a PID.

168. `ps`

Display processes:

```
ps
```

169. `ps aux`

Common process listing:

```
ps aux
```

170. `ps -ef`

Another common format:

```
ps -ef
```

171. Process Fields

Common fields:

```
PID  
PPID  
USER  
CPU  
MEM  
TTY  
STAT  
START  
TIME  
COMMAND
```

172. PID

PID = Process ID.

Each running process has a process identifier.

173. PPID

PPID = Parent Process ID.

Processes can form parent-child relationships.

174. pstree

Display processes as a tree:

```
pstree
```

175. top

Real-time process monitor:

```
top
```

Exit:

```
q
```

176. htop

A more interactive process viewer:

```
htop
```

It may need to be installed separately.

177. Killing Processes

```
kill PID
```

By default, `kill` sends:

```
SIGTERM
```

This requests graceful termination.

178. kill -9

```
kill -9 PID
```

Sends:

```
SIGKILL
```

SIGKILL cannot be caught or handled by the target process.

Use it as a last resort.

179. Common Signals

```
SIGTERM → 15 → graceful termination request  
SIGKILL → 9 → forceful termination  
SIGHUP → 1 → hangup/reload behavior depending on program  
SIGINT → 2 → interrupt  
SIGSTOP → 19 → stop process  
SIGCONT → 18 → continue stopped process
```

180. pkill

Kill processes by name/pattern:

```
pkill firefox
```

Use carefully.

181. pgrep

Find process IDs by name:

```
pgrep firefox
```

182. Foreground and Background

Run normally:

```
command
```

The shell waits for it.

Run in background:

```
command &
```

183. jobs

Show jobs belonging to the current shell:



```
jobs
```

184. Ctrl + Z

Suspends the foreground process.

It normally sends:

```
SIGTSTP
```

185. fg

Bring a job to the foreground:

```
fg
```

Specific job:

```
fg %1
```

186. bg

Continue a stopped job in the background:

```
bg
```

187. nohup

Run a command so it can continue after the terminal/session disconnects in common cases:

```
nohup ./script.sh &
```

188. Sessions and Job Control

A shell manages jobs.

Conceptually:

```
Shell
├─ foreground job
├─ background job
└─ stopped job
```

Job IDs use:

```
%1
%2
```

while process IDs use:

```
1234  
5678
```

Do not confuse them.

189. Process Priority

Linux processes have a scheduling priority relationship represented by nice values.

Start with a specific nice value:

```
nice -n 10 command
```

Change an existing process:

```
renice 10 -p PID
```

Lower nice values generally mean higher scheduling priority, while higher nice values mean the process is more willing to yield CPU time.

Changing priority toward higher privilege may require elevated permissions.



Linux CLI — System Information and Storage

Note — This section is part of Linux CLI Mastery. Source material from the supplied Linux CLI notes is preserved and organized by topic.

190. System Information

uname

```
uname -a
```

Displays kernel/system information.

191. Kernel Version

```
uname -r
```

192. CPU Information

```
lscpu
```

193. Memory Information

```
free -h
```

194. Detailed Memory

```
cat /proc/meminfo
```

195. Disk Space

```
df -h
```

Shows filesystem disk usage.

196. Directory Size

```
du -sh directory/
```

197. Largest Items

Example:

```
du -ah . | sort -h | tail
```

This can help identify large files/directories.

198. Mounted Filesystems

```
mount
```

or:

```
findmnt
```

199. Block Devices

```
lsblk
```

Shows disks, partitions, and block-device relationships.

200. Disk Partition Information

```
sudo fdisk -l
```

Use caution with partition-management tools.

201. File Descriptors

Linux processes interact with files and many resources through file descriptors.

Common descriptors:

```
0 → stdin  
1 → stdout  
2 → stderr
```

Check a process:

```
ls -l /proc/PID/fd
```

Linux CLI — Networking and Remote Access

Note — This section is part of Linux CLI Mastery. Source material from the supplied Linux CLI notes is preserved and organized by topic.

202. Networking Basics

Linux provides many CLI networking tools.

Important commands:

```
ip  
ping  
ss  
curl  
wget  
ssh  
scp  
dig  
nslookup  
traceroute
```

203. ip

Modern Linux networking administration commonly uses:

```
ip
```

Show interfaces:

```
ip addr
```

or:

```
ip a
```

204. IP Routes

```
ip route
```

Shows the routing table.

205. Link Information

```
ip link
```

206. ping

Tests IP-level reachability using ICMP where permitted.

```
ping example.com
```

Stop:

```
Ctrl + C
```

Limit requests:

```
ping -c 4 example.com
```

207. ss

Displays socket information.

```
ss
```

Listening TCP/UDP sockets:

```
ss -tuln
```

Common flags:

```
-t → TCP  
-u → UDP  
-l → listening  
-n → numeric
```

208. curl

Transfers data from or to a URL.

```
curl https://example.com
```

Headers:

```
curl -I https://example.com
```

Download to a file:

```
curl -o page.html https://example.com
```

209. wget

Download resources:

```
wget https://example.com/file.zip
```

210. ssh

Secure Shell.

Connect to a remote machine:

```
ssh username@server
```

Specify a port:

```
ssh -p 2222 username@server
```

211. SSH Keys

Generate an SSH key pair:

```
ssh-keygen
```

Common modern algorithm:

```
ssh-keygen -t ed25519
```

Private key:

```
~/.ssh/id_ed25519
```

Public key:

```
~/.ssh/id_ed25519.pub
```

Never share your private key.

212. ssh-agent

Manages private keys for authentication:

```
ssh-agent
```

Add a key:

```
ssh-add ~/.ssh/id_ed25519
```

213. scp

Securely copy files over SSH.

Local → remote:

```
scp file.txt user@server:/home/user/
```

Remote → local:

```
scp user@server:/home/user/file.txt .
```

214. rsync

Efficient file synchronization.

```
rsync -av source/ destination/
```

Remote:

```
rsync -av project/ user@server:/home/user/project/
```

215. DNS

DNS translates domain names to network information.

Examples:

```
example.com  
↓  
IP address
```

216. dig

Query DNS:

```
dig example.com
```

A record:

```
dig example.com A
```

MX record:

```
dig example.com MX
```

217. nslookup

Another DNS lookup utility:

```
nslookup example.com
```

dig is generally preferred for detailed DNS troubleshooting.

218. hostname

Show hostname:

```
hostname
```

219. hostnamectl

On systems using systemd:

```
hostnamectl
```

Can display and manage hostname-related system information.

220. Package Management

Package management depends on the Linux distribution.

Common ecosystems:

```
Debian/Ubuntu → apt
Fedora/RHEL   → dnf
Arch          → pacman
openSUSE      → zypper
```



Linux CLI — Packages, Services, Scheduling and Archives

Note — This section is part of Linux CLI Mastery. Source material from the supplied Linux CLI notes is preserved and organized by topic.

221. APT

Update package metadata:

```
sudo apt update
```

Upgrade installed packages:

```
sudo apt upgrade
```

Install:

```
sudo apt install package
```

Remove:

```
sudo apt remove package
```

222. APT Search

```
apt search nginx
```

Show package information:

```
apt show nginx
```

223. APT Removal with Configuration

```
sudo apt purge package
```

This can remove package configuration files managed by the package.

224. DNF

Fedora/RHEL-family systems commonly use:

```
sudo dnf install package
```

Update:

```
sudo dnf upgrade
```

Remove:

```
sudo dnf remove package
```

225. Pacman

Arch Linux:

```
sudo pacman -S package
```

Synchronize and upgrade:

```
sudo pacman -Syu
```

Remove:

```
sudo pacman -R package
```

226. Processes vs Services

A process is a running instance of a program.

A service is a background system component typically managed as a long-running unit.

Modern Linux distributions commonly use:

```
systemd
```

for service management.

227. systemctl

Check service status:

```
systemctl status nginx
```

Start:

```
sudo systemctl start nginx
```

Stop:

```
sudo systemctl stop nginx
```

Restart:

```
sudo systemctl restart nginx
```

228. Enable Service at Boot

```
sudo systemctl enable nginx
```

Enable and start immediately:

```
sudo systemctl enable --now nginx
```

229. Disable Service

```
sudo systemctl disable nginx
```

230. List Running Services

```
systemctl --type=service
```

231. Logs with journalctl

View systemd journal:

```
journalctl
```

Current boot:

```
journalctl -b
```

Follow logs:

```
journalctl -f
```

Specific service:

```
journalctl -u nginx
```

232. Scheduled Tasks

Linux supports scheduled execution.

Important tools:

```
cron  
crontab  
at  
systemd timers
```

233. crontab

Edit your cron jobs:

```
crontab -e
```

View:

```
crontab -l
```

234. Cron Format

Typical format:

```
minute hour day-of-month month day-of-week command
```

Example:

```
0 2 * * * /home/user/backup.sh
```

Means approximately:

```
Every day at 02:00
```

235. Cron Wildcards

```
* → every value
```

Example:

```
*/5 * * * *
```

Runs every five minutes.

236. at

Schedule a command for a one-time future execution.

Example:

```
echo "/home/user/script.sh" | at 18:00
```

Availability depends on system configuration/service.

237. Archives

Linux commonly uses:

```
tar  
gzip  
bzip2  
xz  
zip
```

238. tar

Create archive:

```
tar -cf archive.tar files/
```

Extract:

```
tar -xf archive.tar
```

239. Gzip Compression

Create compressed tar archive:

```
tar -czf archive.tar.gz files/
```

Extract:

```
tar -xzf archive.tar.gz
```

240. XZ

```
tar -cJf archive.tar.xz files/
```

Extract:

```
tar -xJf archive.tar.xz
```

241. Bzip2

```
tar -cjf archive.tar.bz2 files/
```

Extract:

```
tar -xjf archive.tar.bz2
```

242. ZIP

Create:

```
zip -r archive.zip directory/
```

Extract:

```
unzip archive.zip
```



243. Compression vs Archive

Important distinction:

```
tar  → archive multiple files
gzip → compress data
zip  → archive + compression format
```

A **.tar.gz** file means:

```
tar archive
+
gzip compression
```

YESHWANTH-GUJJA



Linux CLI — Editors and Shell Configuration

Note — This section is part of Linux CLI Mastery. Source material from the supplied Linux CLI notes is preserved and organized by topic.

244. Text Editors

Common terminal editors:

```
nano  
vim  
vi  
emacs
```

245. Nano

Open:

```
nano notes.txt
```

Common shortcuts:

```
Ctrl + O → save  
Ctrl + X → exit  
Ctrl + W → search  
Ctrl + K → cut line  
Ctrl + U → paste
```

246. Vim

Open:

```
vim notes.txt
```

Important modes:

```
Normal mode  
Insert mode  
Command-line mode
```

Enter insert mode:

```
i
```

Return to normal mode:

```
Esc
```

Save and quit:

```
:wq
```

Quit without saving:



```
:q!
```

247. Vim Basic Navigation

Normal mode:

```
h → left  
j → down  
k → up  
l → right
```

Start of line:

```
0
```

End of line:

```
$
```

Beginning of file:

```
gg
```

End of file:

```
G
```

248. Vim Search

```
/word
```

Next match:

```
n
```

Previous match:

```
N
```

249. Archive and Backup Mental Model

For a directory:

```
Project  
↓  
tar  
↓  
archive  
↓  
gzip  
↓  
compressed archive
```

Example:

```
tar -czf project.tar.gz project/
```

250. Environment Configuration

Shell startup files depend on shell and login/interactive behavior.

For Bash, commonly encountered files include:

```
~/.bashrc  
~/.profile  
~/.bash_profile
```

System-wide configuration may include:

```
/etc/profile  
/etc/bash.bashrc
```

Exact behavior varies by distribution and shell invocation mode.

251. .bashrc

Commonly used for interactive Bash shell configuration.

Example:

```
export EDITOR=nano  
alias ll='ls -lah'
```

After editing, reload:

```
source ~/.bashrc
```

252. source

Execute a file in the current shell:

```
source ~/.bashrc
```

Equivalent Bash shorthand:

```
~/.bashrc
```

Important distinction:

```
./script.sh
```

usually runs the script in a new process/shell context.

```
source script.sh
```

runs it in the current shell environment.

253. Aliases

Create:

```
alias ll='ls -lah'
```

Then:

```
ll
```

View aliases:

```
alias
```

Remove:

```
unalias ll
```

Aliases are shell features, not standalone executable commands.

254. Functions

Bash functions:

```
greet() {  
    echo "Hello $1"  
}
```

Call:

```
greet Yeshwanth
```

Linux CLI — Bash Scripting

Note — This section is part of Linux CLI Mastery. Source material from the supplied Linux CLI notes is preserved and organized by topic.

255. Shell Scripts

A shell script is a text file containing shell commands.

Example:

```
#!/usr/bin/env bash
echo "Hello"
```

Save as:

```
hello.sh
```

256. Shebang

The first line:

```
#!/usr/bin/env bash
```

is a shebang.

It tells the system which interpreter should be used when the script is executed as a program.

257. Executing a Script

Give execute permission:

```
chmod +x hello.sh
```

Run:

```
./hello.sh
```

Alternatively:

```
bash hello.sh
```

The latter explicitly invokes Bash and does not require the executable bit.

258. Script Variables



```
#!/usr/bin/env bash

name="Yeshwanth"

echo "Hello $name"
```

259. Positional Parameters

For:

```
./script.sh one two
```

Inside the script:

```
$0 → script name
$1 → one
$2 → two
```

All arguments:

```
"$@"
```

Number of arguments:

```
$#
```

260. Exit from Script

```
exit 0
```

Success.

Error example:

```
exit 1
```

261. Conditional Statements

```
if [ "$name" = "Yeshwanth" ]; then
    echo "Hello"
else
    echo "Unknown"
fi
```

Modern Bash also supports:

```
if [[ "$name" == "Yeshwanth" ]]; then
    echo "Hello"
fi
```

262. File Tests



Examples:

```
[ -f file.txt ]
```

Regular file.

```
[ -d directory ]
```

Directory.

```
[ -e path ]
```

Exists.

```
[ -r file.txt ]
```

Readable.

```
[ -w file.txt ]
```

Writable.

```
[ -x file.sh ]
```

Executable.

263. Loops

for

```
for file in *.txt; do
    echo "$file"
done
```

while

```
while condition; do
    ...
done
```

until

```
until condition; do
    ...
done
```

264. Command-Line Safety

Always quote variables containing paths or user input:

```
rm -- "$file"
```

rather than:

```
rm $file
```

Quoting prevents unwanted word splitting and glob expansion in many contexts.

265. --

A standalone:

```
--
```

often tells a command that subsequent arguments should be treated as operands rather than options.

Example:

```
rm -- -important.txt
```

This can remove a filename beginning with -.

266. xargs

Build command arguments from standard input.

Example:

```
printf '%s\n' a b c | xargs echo
```

Be careful when filenames contain spaces, quotes, or newlines.

For safe filename handling with `find`, a common pattern is:

```
find . -type f -print0 | xargs -0 command
```

267. sed

Stream editor.

Simple substitution:

```
sed 's/old/new/' file.txt
```

Global replacement on each line:

```
sed 's/old/new/g' file.txt
```

Print selected lines:

```
sed -n '1,10p' file.txt
```

268. awk

Powerful text-processing language.

Example:

```
awk '{print $1}' file.txt
```

Prints the first whitespace-separated field.

Example:

```
awk -F: '{print $1}' /etc/passwd
```

Uses `:` as the field separator.

YESHWANTH-GUJJA

Linux CLI — Advanced CLI Tools and Reference

Note — This section is part of Linux CLI Mastery. Source material from the supplied Linux CLI notes is preserved and organized by topic.

269. diff

Compare files:

```
diff file1.txt file2.txt
```

Useful for configuration and source-code comparison.

270. cmp

Byte-level comparison:

```
cmp file1 file2
```

271. comm

Compare sorted files:

```
comm file1.txt file2.txt
```

The inputs generally need to be sorted.

272. basename

Extract filename from path:

```
basename /home/user/file.txt
```

Result:

```
file.txt
```

273. dirname

Extract directory portion:

```
dirname /home/user/file.txt
```

Result:

```
/home/user
```

274. realpath

Resolve an absolute path:

```
realpath ./notes.txt
```

275. readlink

Inspect symbolic links:

```
readlink linkname
```

276. date

Display date/time:

```
date
```

Format:

```
date '+%Y-%m-%d %H:%M:%S'
```

277. cal

Display a calendar on systems where installed:

```
cal
```

278. uptime

Show how long the system has been running:

```
uptime
```

It also displays load information.

279. whoami

```
whoami
```

Answers:

```
Which user am I currently operating as?
```

280. hostname

```
hostname
```

Answers:

```
What is this machine called?
```

281. id

```
id
```

Answers:

```
What are my UID, GID, and groups?
```

282. pwd

```
pwd
```

Answers:

```
Where am I?
```

283. ls

```
ls
```

Answers:

```
What is here?
```

284. cd

```
cd directory
```

Answers:

```
Move me somewhere else.
```

285. cp

```
cp source destination
```

Answers:

```
Copy this.
```

286. mv

```
mv source destination
```

Answers:

```
Move/rename this.
```

287. rm

```
rm file
```

Answers:

```
Remove this.
```

288. mkdir

```
mkdir directory
```

Answers:

```
Create this directory.
```

289. touch

```
touch file
```

Answers:

```
Create this file if absent / update its timestamps.
```

290. cat

```
cat file
```

Answers:

```
Show this file.
```

291. grep

```
grep pattern file
```

Answers:

```
Find matching text.
```

292. find

```
find path ...
```

Answers:

```
Find filesystem objects according to conditions.
```

293. ps

```
ps
```

Answers:

```
What processes are running?
```

294. kill

```
kill PID
```

Answers:

```
Send a signal to this process.
```

295. df

```
df -h
```

Answers:

```
How much filesystem space is available?
```

296. du

```
du -sh directory
```

Answers:

```
How much space is this directory using?
```

297. free

```
free -h
```

Answers:

```
How much memory is available?
```

298. ip

```
ip addr
```

Answers:

```
What network interfaces and addresses exist?
```

299. ss

```
ss -tuln
```

Answers:

```
What sockets are listening?
```

300. systemctl

```
systemctl status service
```

Answers:

```
What is the state of this systemd service?
```

Linux CLI — Troubleshooting, Interview and Learning

Note — This section is part of Linux CLI Mastery. Source material from the supplied Linux CLI notes is preserved and organized by topic.

301. Essential Command Cheat Sheet

YESHWANTH-GUJJA



```
pwd
ls
ls -la
cd
cd ..
cd ~
mkdir
mkdir -p
touch
cp
cp -r
mv
rm
rm -r
cat
less
head
tail
tail -f
find
grep
sort
uniq
cut
sed
awk
wc
file
stat
ln
ln -s
chmod
chown
chgrp
umask
whoami
id
groups
ps
ps aux
top
kill
pkill
pgrep
jobs
fg
bg
df -h
du -sh
free -h
uname -a
lscpu
ip addr
ip route
ping
ss
curl
wget
ssh
scp
rsync
dig
systemctl
journalctl
crontab
tar
zip
unzip
man
history
```

302. Linux CLI Troubleshooting Workflow

When something goes wrong, use a structured process.



Step 1 — Identify where you are

```
pwd
```

Step 2 — Inspect the directory

```
ls -la
```

Step 3 — Identify the file

```
file filename
```

Step 4 — Check permissions

```
ls -l filename
```

Step 5 — Check ownership

```
stat filename
```

Step 6 — Check the process

```
ps aux | grep program
```

Step 7 — Check logs

```
journalctl -u service
```

or:

```
tail -f /var/log/application.log
```

Step 8 — Check networking

```
ip addr  
ip route  
ping host  
ss -tuln
```

Step 9 — Check disk

```
df -h  
du -sh directory
```

Step 10 — Check documentation

```
man command
```

303. Linux CLI Interview Questions

Fundamentals

What is CLI?

A command-line interface allows users to interact with a computer through textual commands.

What is a shell?

A shell interprets commands and executes them.

What is Bash?

Bash is the Bourne Again SHell, a widely used Unix/Linux shell.

Terminal vs shell?

A terminal provides an interface; the shell interprets commands inside it.

304. Filesystem Questions

What is `/`?

The root of the Linux filesystem.

What is `/home`?

Typically contains normal users' home directories.

What is `/etc`?

System configuration.

What is `/var`?

Variable application/system data such as logs and caches.

What is `/tmp`?

Temporary data.

What is `/proc`?

A virtual filesystem exposing process/kernel information.

305. Permissions Questions

What does `755` mean?

```
rwxr-xr-x
```

Owner:

```
rwX
```

Group:

```
r-x
```

Others:

```
r-x
```

What does 644 mean?

```
rw-r--r--
```

What does chmod do?

Changes permissions.

What does chown do?

Changes ownership.

What does umask do?

Controls which permissions are masked when new files/directories are created.

306. Process Questions

What is PID?

Process ID.

What is PPID?

Parent Process ID.

kill vs kill -9?

`kill` normally sends SIGTERM; `kill -9` sends SIGKILL.

Why should SIGKILL not be the first choice?

SIGTERM allows the process to clean up and exit gracefully; SIGKILL cannot be handled by the target.

307. Networking Questions

What does ip addr show?

Network interfaces and IP addresses.

What does ip route show?

Routing information.

What does ss show?

Socket information.

What does curl do?



Transfers data using URLs and is widely used for HTTP/API interaction.

What does SSH do?

Provides secure remote shell access and other secure transport functionality.

308. Storage Questions

df vs du

```
df → filesystem free/used space
du → space consumed by files/directories
```

lsblk

Shows block devices and their relationships.

309. Shell Questions

What does \$? mean?

Exit status of the most recently executed foreground pipeline/command context.

What does \$0 mean in a script?

The script/command name as received by the shell.

What does \$1 mean?

First positional argument.

What does \$# mean?

Number of positional arguments.

What does \$@ mean?

The positional parameters, especially when used as "\$@" to preserve argument boundaries.

310. Redirection Questions

What is >?

Redirect stdout and overwrite the target file.

What is >>?

Redirect stdout and append.

What is 2>?

Redirect stderr.

What is |?

Pipe stdout from one command to stdin of another.



What is `/dev/null`?

A special device that discards written data.

311. Practical Command Chains

Find errors in a log:

```
grep -i "error" application.log
```

Count them:

```
grep -i "error" application.log | wc -l
```

Count unique error lines:

```
grep -i "error" application.log | sort | uniq -c
```

Find large files:

```
find . -type f -size +100M
```

Find a process:

```
ps aux | grep nginx
```

Find listening services:

```
ss -tuln
```

Check disk:

```
df -h
```

Check memory:

```
free -h
```

Check service logs:

```
journalctl -u nginx
```

312. The Linux CLI Mental Model

The most important conceptual model is:

```
User
↓
Terminal
↓
Shell
↓
Command
↓
Process
↓
Kernel
↓
Hardware / Filesystem / Network
```

A command generally:

```
Input
↓
Shell parsing
↓
Expansion
↓
Program execution
↓
stdin / stdout / stderr
↓
Exit status
```

313. Linux CLI Learning Order

For learning and interviews, master the concepts in this order:



```
1. Terminal
2. Shell
3. Bash
4. Command syntax
5. pwd
6. ls
7. cd
8. mkdir
9. touch
10. cp
11. mv
12. rm
13. cat
14. less
15. head / tail
16. find
17. grep
18. pipes
19. redirection
20. stdin/stdout/stderr
21. variables
22. environment variables
23. PATH
24. permissions
25. chmod
26. chown
27. users/groups
28. sudo
29. processes
30. ps
31. top
32. kill
33. jobs
34. systemctl
35. journalctl
36. disk management
37. networking
38. SSH
39. package management
40. archives
41. Bash scripting
42. text processing
43. automation
44. troubleshooting
```

314. Final Linux CLI Reference

If you remember only the fundamental commands initially, remember this set:



```
pwd                # Where am I?
ls -la             # What is here?
cd                 # Move
mkdir              # Create directory
touch              # Create/update file
cp                 # Copy
mv                 # Move/rename
rm                 # Remove
cat                # Display
less              # Read interactively
head               # Beginning
tail               # End
find               # Find files
grep               # Find text
sort               # Sort
uniq               # Deduplicate
wc                 # Count
cut                # Extract fields
sed                # Transform text
awk                # Process structured text
chmod              # Permissions
chown              # Ownership
whoami             # Current user
id                 # Identity/groups
sudo               # Elevated command
ps                 # Processes
top                # Process monitor
kill               # Send signal
df -h              # Filesystem space
du -sh             # Directory space
free -h            # Memory
ip addr            # Network addresses
ip route           # Routes
ss -tuln           # Listening sockets
ping               # Connectivity test
curl               # HTTP/data transfer
ssh                # Remote shell
scp                # Secure copy
rsync              # Synchronization
systemctl          # Services
journalctl         # Systemd logs
tar                # Archives
man                # Documentation
history            # Command history
```

The central skill is not memorizing hundreds of commands. It is understanding how commands, arguments, options, paths, permissions, processes, pipes, redirection, environment variables, and exit statuses interact. Once that model is clear, the Linux CLI becomes much easier to learn and troubleshoot.

Linux CLI Mastery — Deep Dives and Gaps Filled

This file contains important CLI-adjacent Linux concepts that deserve a standalone treatment beyond the original command-by-command notes.

1. Shell Parsing Order

A useful Bash mental model is:

```
input line
↓
lexing / tokenization
↓
quotes and escapes
↓
parameter expansion / command substitution / arithmetic expansion
↓
pathname expansion (globbing)
↓
redirection setup
↓
command lookup
↓
execution
```

The exact shell grammar is more nuanced, but this model explains many practical surprises.

2. Shell Builtins vs External Commands

Use `type`, `command -V`, or `help` to distinguish them:

```
type cd
type echo
type grep
help cd
```

Builtins can modify the current shell state. That is why `cd` must be a shell builtin (or otherwise executed in the current shell) to change the caller's working directory.

3. Login, Interactive and Non-Interactive Shells

Do not assume every Bash startup file is read in every situation. The files and order depend on whether Bash is a login shell, interactive shell, or non-interactive shell, and distribution conventions can add system-wide files.

Useful checks:

```
echo "$0"
shopt -q login_shell; echo $?
ps -p $$ -o pid,ppid,args
```

4. /proc as a Process Interface

Useful examples:

```
cat /proc/$$/cmdline
tr '\0' ' ' < /proc/$$/cmdline; echo
cat /proc/$$/status
readlink -f /proc/$$/cwd
readlink -f /proc/$$/exe
ls -l /proc/$$/fd
```

This makes the relationship between processes, file descriptors, executable images, and the kernel more concrete.

5. Filesystem Mounts and `/etc/fstab`

A filesystem can be attached to the directory tree through a mount point. Inspect mounts with:

```
findmnt
findmnt /
lsblk -f
```

`/etc/fstab` describes filesystems that may be mounted automatically. Prefer stable identifiers such as UUIDs rather than assuming device names will never change:

```
blkid
```

Never edit or apply mount configuration blindly on a production machine. Validate with `mount -a` in an appropriate maintenance context.

6. LVM Mental Model

Logical Volume Manager commonly follows:

```
physical disks / partitions
    ↓
physical volumes (PV)
    ↓
volume group (VG)
    ↓
logical volumes (LV)
    ↓
filesystem
    ↓
mount point
```

Useful inspection commands:

```
pvs
vgs
lvs
lsblk
```

Resizing storage is filesystem-specific; expanding a block device and expanding the filesystem are separate operations.

7. ACLs

Traditional Unix permissions are not always enough for fine-grained access. POSIX ACLs can provide additional user/group entries.

```
getfacl file.txt
setfacl -m u:alice:r file.txt
```

An ACL does not replace the need to understand the owner/group/mode bits; it extends the access-control model.

8. Linux Capabilities

Capabilities split some root-like privileges into smaller units. This matters when a program needs a specific privileged operation without unrestricted root authority.

Inspect process capabilities with tools such as:

```
getcap /path/to/program
capsh --print
```

Treat capability changes as security-sensitive configuration.

9. SELinux and AppArmor

Linux access control can include mandatory access-control frameworks in addition to traditional mode bits. Common examples are SELinux and AppArmor.

Do not conclude that a file is accessible merely because `ls -l` looks correct. A security policy can still deny the operation.

Useful discovery commands vary by distribution, for example:

```
getenforce
sestatus
apparmor_status
```

10. Systemd Unit Mental Model

`systemctl` is not only about services. `systemd` manages units such as services, sockets, mounts, timers, targets, and paths.

```
systemctl list-units
systemctl list-unit-files
systemctl cat nginx.service
systemctl show nginx.service
```

When debugging a service, distinguish:

```
unit configuration
→ dependency ordering
→ process startup
→ runtime logs
→ resource/security restrictions
```

11. Resource Limits

Per-process resource limits can explain failures that look like application bugs.

```
ulimit -a
cat /proc/$$/limits
```

Common limits include open files, processes, core dumps, and address-space-related limits.

12. Open Files and Sockets

`lsof` can connect processes to files, devices, and sockets:

```
lsof -p PID
lsof -i :8080
```

This is often useful when a port is unexpectedly occupied or a file cannot be unlinked/umounted because a process still holds it open.

13. /etc/hosts, /etc/resolv.conf and Name Resolution

Useful files for diagnosing name-resolution behavior include:



```
cat /etc/hosts
cat /etc/resolv.conf
getent hosts example.com
```

getent is particularly useful because it asks the system's configured name-service mechanisms rather than only querying DNS directly.

14. HTTP Troubleshooting with **curl**

A practical progression is:

```
curl -I https://example.com
curl -v https://example.com
curl -sS https://example.com
curl -sS -o /dev/null -w '%{http_code}\n' https://example.com
```

Separate DNS failure, TCP connection failure, TLS failure, HTTP status failure, and application-level response problems instead of treating all of them as a generic 'network issue'.

15. Secure Bash Defaults

For scripts where strict behavior is appropriate:

```
set -Eeuo pipefail
```

Use this intentionally rather than mechanically. Understand how **set -e** interacts with conditionals, pipelines, command substitutions, and expected non-zero statuses.

A safer script also validates input, quotes expansions, uses **--** where applicable, avoids parsing **ls**, and cleans up temporary resources.

16. Temporary Files

Prefer **mktemp** instead of predictable temporary filenames:

```
tmpdir=$(mktemp -d)
trap 'rm -rf -- "$tmpdir"' EXIT
```

This reduces collisions and common temporary-file race problems.

17. Exit Status and Pipelines

A pipeline's status has subtle semantics. Bash can expose the status of each command with **PIPESTATUS**, and **pipefail** changes the pipeline's aggregate status:

```
false | true
echo "$?"

set -o pipefail
false | true
echo "$?"
```

Understand this before writing automation that treats a pipeline as a single success/failure unit.

18. **xargs** vs **find -exec**

Prefer **find -exec ... {} +** or null-delimited **xargs -0** when handling arbitrary filenames. Never assume filenames cannot contain spaces, tabs, quotes, or newlines.

19. Common Production Investigation Matrix



Symptom	First checks	Next checks
Command not found	<code>type, command -v, echo \$PATH</code>	shell startup files, package install
Permission denied	<code>id, ls -l, namei -l</code>	ACLs, SELinux/AppArmor, parent directories
Disk full	<code>df -h</code>	<code>du</code> , deleted-open files via <code>lsof</code>
Port unavailable	<code>ss -ltnp</code>	<code>lsof -i :PORT</code> , service config
Service failed	<code>systemctl status</code>	<code>journalctl -u, systemctl cat</code>
DNS issue	<code>getent hosts, resolvectl</code> where available	<code>dig</code> , resolver configuration
High CPU	<code>top, ps</code>	process-specific profiling/logs
High memory	<code>free, ps, top</code>	process limits, OOM evidence
SSH failure	<code>ssh -v</code>	keys, permissions, server logs, network path

YESHWANTH-GUJJA



Appendix — Linux CLI Cheatsheet

Linux CLI Cheat Sheet

Navigation

```
pwd
ls -la
cd /path
cd ..
cd ~
```

Files

```
mkdir -p dir/subdir
touch file
cp source destination
cp -r dir destination
mv source destination
rm file
rm -r dir
file path
stat path
ln -s target link
```

Search and text

```
find . -type f -name '*.log'
grep -Rni 'error' .
head -n 20 file
tail -n 20 file
tail -f file
sort file | uniq -c
cut -d: -f1 /etc/passwd
sed 's/old/new/g' file
awk '{print $1}' file
```

Streams

```
command > out.txt
command >> out.txt
command 2> err.txt
command > out.txt 2>&1
command | grep pattern
command > /dev/null 2>&1
```

Identity and permissions

```
whoami
id
groups
chmod 755 script.sh
chmod u+x script.sh
sudo chown user:group file
umask
```

Processes

```
ps aux
top
pgrep name
pkill name
kill PID
kill -TERM PID
kill -KILL PID
jobs
fg %1
bg %1
```

System and storage

```
uname -a
lscpu
free -h
df -h
du -sh directory
lsblk
findmnt
```

Networking

```
ip addr
ip route
ping -c 4 host
ss -tuln
curl -I https://example.com
curl -v https://example.com
dig example.com
ssh user@host
scp file user@host:/path/
rsync -av source/ user@host:/path/
```

Services and logs

```
systemctl status service
systemctl start service
systemctl enable --now service
journalctl -u service
journalctl -b
journalctl -f
```

Packages

```
sudo apt update && sudo apt upgrade
sudo apt install package
sudo dnf install package
sudo pacman -S package
```

Archives

```
tar -czf archive.tar.gz directory/
tar -xzf archive.tar.gz
zip -r archive.zip directory/
unzip archive.zip
```

Bash



```
name=value
echo "$name"
export NAME=value
echo "$?"
echo "$#"
printf '%s\n' "$@"
command1 && command2
command1 || command2
$(command)
```

YESHWANTH-GUJJA

Appendix — Practice Examples

Small reproducible exercises from the supplied repository.

01 Filesystem

Filesystem Practice

Run in a disposable directory:

```
mkdir -p linux-cli-lab/{docs,logs,archive}
cd linux-cli-lab
printf '%s\n' one two three > docs/items.txt
cp docs/items.txt docs/copy.txt
mv docs/copy.txt archive/copy.txt
find . -type f -print
```

Clean up with `cd .. && rm -rf -- linux-cli-lab` only when you have verified the path.

02 Pipes Redirection

Pipes and Redirection Practice

```
printf '%s\n' error info error warning | sort | uniq -c
printf '%s\n' alpha beta gamma > output.txt
grep error output.txt > errors.txt 2> /tmp/example-errors.txt
```

Focus on which stream each operator affects.

03 Permissions

Permissions Practice

```
printf '#!/usr/bin/env bash\nnecho hello\n' > hello.sh
ls -l hello.sh
chmod u+x hello.sh
./hello.sh
chmod 644 hello.sh
```

Do not experiment with ownership or recursive permissions on system directories.

04 Processes

Process Practice

```
sleep 300 &
jobs
pgrep -a sleep
ps -o pid,ppid,stat,cmd -p $(pgrep -n sleep)
kill -TERM $(pgrep -n sleep)
```

05 Networking

Networking Practice



```
ip addr
ip route
ss -tuln
ping -c 2 example.com
curl -I https://example.com
dig example.com
```

These commands inspect local state and public connectivity; results depend on your network.

06 Services Logs

Services and Logs Practice

On a systemd-based Linux system:

```
systemctl --type=service --state=running
journalctl -b -p warning
```

Use an installed, non-critical service for further experiments.

07 Bash Scripting

Bash Scripting Practice

Run:

```
chmod +x hello.sh
./hello.sh Yeshwanth
```

Then modify it to validate arguments, check files, and return meaningful exit statuses.

Example script — hello.sh

```
#!/usr/bin/env bash
set -Eeuo pipefail

name=${1:-World}
printf 'Hello, %s\n' "$name"
```